

SmartCare v0.3 - Domain Model Workbook

Week 6 student resource

Requirement-to-Concept Trace

Requirement	Concept	State/behaviour	Decision
FR-01: Patient Registration	Patient	State: patient_id, full_name, phone, dob, address Behavior: Instantiation & validation	Accepted. Core domain entity required to hold demographic data and anchor bookings.
FR-02: Patient Contact Updates	Patient	Behavior: update_details(phone, address)	Accepted. Encapsulates mutability cleanly without altering primary key (patient_id).
FR-03: Practitioner Profile	Practitioner	State: practitioner_id, doctor_name, speciality, room_number	Accepted. Core domain entity providing medical resource details.
FR-04: Appointment Booking	Appointment	State: appointment_id, patient_id, practitioner_id, date_time Behavior: create_booking()	Accepted. Core transactional class representing the 1-to-1 association at a specific time.
FR-05: Double-Booking Prevention	Appointment	Behavior: check_collision()	Accepted. Class method validates practitioner time availability against active bookings before creation.
FR-06: Controlled Status Transitions	Appointment	State: status (AppointmentStatus) Behavior: update_status()	Accepted. Enforces finite state machine (SCHEDULED -> ARRIVED -> IN_PROGRESS -> COMPLETED).
FR-07: Mandatory Cancellation Reason	Appointment	State: cancellation_reason Behavior: cancel_appointment(reason)	Accepted. Explicit method enforcing a non-empty reason string when moving to CANCELLED.
FR-08: Historical Preservation	Patient / Appointment	State: appointment_history Behavior: add_to_history(), is_active()	Accepted. Objects are retained in history collections rather than being hard-deleted.
FR-09: Daily Schedule Query	Practitioner	Behavior: get_daily_schedule(date)	Accepted. Filters assigned active appointments by YYYY-MM-DD string prefix.
FR-10: Patient History Query	Patient	Behavior: get_appointment_history()	Accepted. Returns a copy of the patient's past and present booking records.

FR-11: Appointment ID Lookup	System Controller / Registry	Behavior: get_appointment_by_id(id)	Accepted. Adds direct dictionary/registry lookup capability using unique booking IDs.
FR-12: Unregistered Entity Check	Appointment / Registry	Behavior: Foreign key lookup in create_booking()	Accepted. Validates that patient_id and practitioner_id exist in registered registries prior to instantiation.

CRC Cards

Patient

Responsibilities	Collaborators
Maintain patient ID, contact information, and personal details.	Appointment (stores references to historical and active booking objects)
Maintain a historical log of all past and present appointments linked to the patient.	Receptionist (invokes registration and contact updates)

Practitioner

Responsibilities	Collaborators
Maintain practitioner profile details, medical specialty, and allocated consultation room number.	Appointment (inspects assigned booking dates, times, and active statuses)
Filter and return assigned daily appointment schedules for a requested date.	Receptionist (queries practitioner availability and schedules)

Appointment

Responsibilities	Collaborators
Enforce booking collision checks for a practitioner and control valid lifecycle status transitions.	Patient (validates patient reference and updates patient booking history).
Enforce cancellation rules by mandating a non-empty cancellation reason text when moving to the cancelled state.	Practitioner (validates practitioner reference and checks room/time schedule availability).

Optional class: ClinicController

Responsibilities	Collaborators
Maintain system registries (patients, practitioners, appointments) and execute direct lookups by ID (satisfying FR-11 and FR-12).	Patient (validates existence and invokes add_to_history()) Practitioner (validates existence, checks schedule availability, and invokes assign_appointment())
Orchestrate the appointment booking lifecycle by validating entity existence, enforcing collision checks, and linking confirmed bookings to patient histories and practitioner schedules.	Appointment (invokes create_booking() factory method and state update operations)

UML Class Diagram

Insert/draw UML here. Include defensible relationships and multiplicities.

SEE SEPARATE DIAGRAM ATATCHMENT IN THE SAME FOLDER

Design Rationale

Explain class selection, responsibility allocation and key relationships.

Class Selection

The system uses a four-class model designed around clear domain boundaries and responsibility separation:

- **Patient:** Represents the registered individual receiving care.
- **Practitioner:** Represents the healthcare professional providing care.
- **Appointment:** Represents the scheduled consultation connecting a patient to a practitioner at a specific date and time.
- **ClinicController:** Acts as the aggregate root/system orchestrator managing central lookup registries and coordinating higher-level administrative workflows.

Responsibility Allocation

- **Information Expert:**
 - **Patient** holds patient demographics and protects its own appointment history.
 - **Practitioner** manages practitioner details and filters its own daily schedule.
 - **Appointment** manages lifecycle states (SCHEDULED, ARRIVED, IN_PROGRESS, COMPLETED, CANCELLED), enforces valid state transitions, and validates cancellation reasons.
- **Controller:**
 - **ClinicController** handles incoming system calls (registration, booking, queries), validates foreign entity existence (FR-12), checks collision availability, and links bookings to respective patient and practitioner collections.

Key Relationships & Multiplicities

- **ClinicController->Domain Entities (Aggregation):**
 - ClinicController holds collections (0..*) of Patient, Practitioner, and Appointment objects as the central source of truth for the application.
- **Patient -> Appointment (1-to-Many Association):**
 - Multiplicity: 1 Patient to 0..* Appointments.
 - A patient can have multiple historical or upcoming bookings, but deleting or cancelling an appointment does not delete the patient record.
- **Practitioner -> Appointment (1-to-Many Association):**
 - Multiplicity: 1 Practitioner to 0..* Appointments.
 - A practitioner maintains an assigned list of appointments, but the collision logic prevents two active bookings from sharing the exact same time slot.
- **Appointment -> AppointmentStatus (Dependency):**
 - Appointment uses the AppointmentStatus enumeration to validate and enforce its internal finite state machine lifecycle.

AI Design Review Record

AI suggestion	Evidence	Decision	Reason	Model change
---------------	----------	----------	--------	--------------

Creation of a ClinicController Class	Administrative workflows require validating that patient_id and practitioner_id exist in the system prior to creating a booking	Accepted	Provides a single, centralized registry for entity lookup and guarantees referential integrity before creating bookings.	Add a new ClinicController class as an aggregate root container containing _patients: dict, _practitioners: dict, and _appointments: dict.